

Document number: P0679R0

Date: 2017-06-14

Author: Torvald Riegel riegel@redhat.com

Audience: SG1

Forward progress vs. futures and continuations

In what follows, I will discuss issues that arise in the combination of weaker-than-concurrent forward progress guarantees with futures and continuations. These issues are a blocker for the Executors proposal, for example. I do not propose concrete solutions, but I describe design choices we need to make (i.e., the authors of the affected proposals and SG1).

The problem

A future is often described as a handle to work that might not have completed yet. Waiting for this work to finish is a blocking operation. Thus, to give that blocking operation a chance to complete eventually, something needs to ensure that the work that is being waited on is actually executed eventually.

This point doesn't require much further attention in the current version of the standard: Whenever a future is the only handle to work that is spawned, the work is guaranteed to execute eventually; `std::async` creates threads of execution (TOEs) that are equivalent to `std::thread`, so the work associated with a future is always guaranteed to have concurrent forward progress. (Remember that TOEs are not OS threads, but can have concurrent, parallel, or weakly parallel forward progress.)

However, this is not true anymore if considering other proposals or the Concurrency TS:

- `future::then()` in the Concurrency TS does not specify what the forward progress guarantee for the continuation is. [LWG 2533](#) discusses related issues, in particular which TOE a continuation may execute on, but it does not consider forward progress (e.g., of TOEs managed implicitly by the implementation).
- The executors proposal (P0443) contains execution functions that return futures that are the only handle to the TOEs created by the executor. They have weaker-than-concurrent forward progress currently by default, so they are not actually guaranteed to get started unless that is enforced through some other means.
- Asynchronous parallel algorithms (P0361R1) provides variants of the parallel algorithms where the returned future is the only handle to the to-be-executed, weaker-than-concurrent-progress work. This creates a similar problem as in the case of executors. Notably, this breaks (or circumvents) how *blocking with progress guarantee delegation* takes care of the problem in the parallel algorithms that we'll have in C++17.

Note that a future can also be used as purely a synchronization mechanism (which I'll discuss further below). Ambiguity between this synchronization mechanism usage and using a future as handle for work is a large part of the problem, I think.

What we can do about it

When we specified forward progress properly, this consisted of two parts (1) weaker-than-concurrent progress guarantees, and (2) a mechanism to ensure that programs will execute eventually despite the weaker-than-concurrent progress guarantees for some parts of a program. We called the latter blocking with progress guarantee delegation in the standard (see 1.10.2p14), but I'll use the internal short-hand *boost-blocking* in what follows.

In a nutshell, the notion of boost-blocking is a way to specify that an operation blocked on other work will do its best to get that other work executed, if the operation itself is guaranteed to make progress eventually at the particular point in time. (This is purely considering the implementation scheduler's perspective, so even if the operation is still blocked, the scheduler can allow it to make progress -- or not.)

As an example, let us assume that `main()` executes a parallel for-each with the par execution policy. The for-each will conceptually spawn TOEs (which do the actual work) with parallel forward progress. Parallel progress means that such a TOE is *not* required to get started -- but once it is started, it will always be allowed to make progress eventually. This would be nonsensical if considered in isolation because nothing is required to start the TOEs. However, the for-each is also specified to be a boost-blocking operation. Therefore, the implementation is required to delegate the strong concurrent progress of the TOE executing `main()` to some of the weaker-progress TOEs that the for-each is blocked on, until all of them have finished execution.

This allows the implementation to choose the level of parallelism in the execution that makes sense on the particular hardware. There will be at least one concurrent TOE running, but likely many TOEs will run in parallel. This approach has several advantages:

- If a program needs a result, it will get computed. Logical dependencies drive computation.
- This is all transparent for the program, and programs do not have to manage or allocate compute resource explicitly (such as wondering how many OS threads a thread pool should have). Thus, it also makes it easier to share compute resources between different parts of a program.
- It allows the implementation to do the right thing, for example if the number of available compute resources varies during the runtime.
- It allows for on-demand execution.

There are different ways for the implementation to satisfy boost-blocking. For example, it can execute work that is queued up for execution but not executed by another TOE yet in the boost-blocking operation itself. Or it can use a simple bounded thread pool and, if work that the boost-blocking operation depends on hasn't been executed after a minute, add another OS thread to the thread pool. Or it can inspect the set of TOEs and associated work items currently being executed, and simply do nothing more if the boost-blocking operation waits for any of these active TOEs. And so on.

Thus, the solution is to specify that certain operations are boost-blocking operations. The decisions that need to be made are which operations these should be.

Design decisions we need to make

The problem surfaces in the use of futures, in particular when their waiting functions could both be intended to be a synchronization mechanism and a handle for future work. By synchronization mechanism, I mean something like a binary semaphore, a latch, or a pair of unlock/lock calls on a mutex; a future can be used to a similar effect. In these cases, we simply assume that the providing side of the synchronizing pair of TOEs will be executed eventually (otherwise, we'd likely have a deadlock in our program).

We could treat futures as just a synchronization mechanism -- even though `std::async`'s specification already shows that this is not quite practical for other reasons besides forward progress. If we would do it, we would need to add boost-blocking somewhere else, so that it is ensured that the weaker-than-concurrent work is started (for parallel progress) or seen through until the end (for weakly parallel progress).

For example, we could associate all work with a thread pool or work queues, and specify boost-blocking for some operation of the thread pool or work queues. These could be new operations (P0443R1 had some on the thread pool to "drain" all remaining work) or a change to existing ones, such as adding boost-blocking to the destructor.

However, while technically doable, I do *not* think that we should follow that approach. Doing so loses the advantages behind boost-blocking, in particular that dependencies on other work drive execution of that work (see above for more advantages). This is important because otherwise, programmers have to think about the synchronization used for dependencies *and* about providing compute resources that these dependencies can be fulfilled -- and these two things are separate and one needs separate APIs for them. Doing both in one go is much simpler.

Also, the dependencies are useful feedback to the implementation's scheduler of work (e.g., the code that prioritizes work queued up for a implementation-internal thread pool); for example, they are an important aspect of the critical path.

Furthermore, if boost-blocking is tied to execution resources (e.g., a thread pool), then the granularity of boost-blocking, and thus the dependency, will likely often be more coarse-granular than necessary: We want to share resources, so it should be easy to let distinct operations with parallelism share the same underlying resources. I don't think we want to abuse resource abstraction such as thread pools to convey progress dependencies.

Thus, we need to decide which blocking operations should be specified as providing boost-blocking.

(Note that there is some interaction between boost-blocking and the thread identity guarantees we give: the stricter the latter, the less leeway implementations of boost-blocking have. See LWG 2533 for a related SG1 discussion that covers the thread identity choices. Nonetheless, I think that enabling dependencies to drive computation is more important than narrow requirements for TLS, and that the gains of narrow requirements probably don't justify the added implementation complexity (and runtime overheads).)

`future::wait()` and `future::then()`

If we want to be able to use something like `std::async` and create weaker-than-concurrent TOEs (e.g., because of using a certain executor), we need boost-blocking somewhere in the operations of the handle to this work (i.e., in the future).

Specifying `future::wait()` to boost-block would accomplish that (and similarly for `wait_for` and `wait_until`). It probably adds implementation complexity, primarily in that there needs to be some way to figure out whether the providing side can be boosted. It may not always be possible to do that, so perhaps this would only apply to certain kinds of futures.

We might also add a `future::wait_and_do()` function that is, unlike `wait()` in this case, specified to boost-block. That would allow programs to still use futures as purely a synchronization mechanism and thus avoid potential runtime overheads or effects on TLS of the blocked TOE. However, we'd still need to figure out what can actually be boosted (as in the previous case).

`future::then()` is also affected: If the continuation is requested to have stronger forward progress than the TOE that it waits on, should `then()` be a boost-blocking operation similarly to `wait()`? I think the answer depends on the programmer's intent:

- If the intent is to simply trigger the continuation when something happens (i.e., when the future is made ready), then it is not necessary (and perhaps even incorrect) to specify `then()` to be boost-blocking. For example, if one wants to start a watchdog TOE as soon as something happens, then one might need that TOE to have concurrent progress guarantees -- but this need does not mean that the precondition has to be computed.
- In contrast, if the continuation is something that the program wants to always run, then `then()` conveys a dependency in the program, and it should be specified to be boost-blocking. This would be a safer choice than the currently specified behavior, in that it strengthens requirements on the implementation without allowing additional behavior.

Both use cases seem to be sensible. Perhaps having two variants of `then()` would be useful, similar to the `wait()` case: The first example is similar to a synchronization use case (i.e., trigger something to happen), whereas the second is a dependency.

More generally speaking, do we need different kinds of futures? We've been speaking about a future concept for a while now, and the difference between the synchronization-mechanism usage and a handle to work may be yet another reason for future differentiation. Alternatively, we could say that futures are always a handle to work, and waiting on the result would be always boost-blocking.

The Executor proposal's two-way asynchronous execution functions

In case of these execution functions (e.g., `bulk_async_execute`), the returned future is always a handle to the parallel work. P0443R1 provides a mechanism to wait for outstanding work on a thread pool, but this suffers from the issues I explained previously (e.g., abusing a resource abstraction as a handle to work, and the granularity-of-waiting problems that this results in). Furthermore, while the thread pool has such a mechanism, there is no general way to boost-block on the spawned work for arbitrary executors. Therefore, like in the `std::async` example, the `wait()` member function of the returned futures should be specified to perform boost-blocking.

On a similar note, the one-way asynchronous executions functions are problematic in that they don't return a handle to the work, so an implementation is allowed to ignore these requests for execution unless they happen to end up on an instance of the thread pool specified in P0443R1. There must be a generic way to boost-block on these execution functions; if there isn't, they should not be provided.

Asynchronous parallel algorithms

This is similar to the case of the Executors proposal's asynchronous execution functions. Additionally, the asynchronous parallel algorithms can be executed without supplying an executor, so if `future::wait()` does not boost-block, there is no other way to enforce execution of the parallel work.